

Введение в NumPy: Числовой Python

Сара записывает оценки своего второго класса в электронную таблицу. Ее веб-браузер записывает, что она посещала эту таблицу, а также все остальные посещенные ею сайты. Эти сайты фиксируют ее местонахождение, время, которое она провела на них, и места, которые она посетила в следующий раз. Мир полон всевозможных наборов данных, и изучение того, как создавать, анализировать и манипулировать этими наборами данных, может дать нам некоторое представление и контроль над нашим цифровым окружением.

В этом уроке мы будем создавать наборы данных с одной переменной и управлять ими. Один из способов представить себе набор данных с одной переменной - это то, что он содержит ответы на вопрос. Например, мы можем спросить 100 человек: «Какой у вас рост?» Их высота в дюймах будет составлять наш набор данных.

Для работы с нашими наборами данных мы будем использовать мощный модуль Python, известный как NumPy, что означает числовой Python.

NumPy имеет множество применений, в том числе:

- Эффективная работа с большим количеством номеров одновременно
- Генерация случайных чисел
- Выполнение множества различных числовых функций (например, вычисление $\sin$, $\cos$, $\tan$, mean , median и т. Д.)

В следующих упражнениях мы узнаем, как создавать одномерные и двумерные массивы и выполнять основные операции с массивами.

Задание

Изучите код NumPy, расположенный ниже (Работа с NumPy). Это несколько примеров того, что вы узнаете в ходе урока, и того, как NumPy может быть невероятно полезным и мощным инструментом. Не волнуйтесь, если вы не знаете, как работает каждый фрагмент кода; это просто дает вам представление о том, что вы можете делать с NumPy!

Работа с NumPy

NumPy отлично подходит для хранения и обработки числовых данных в массивах.

Давайте посмотрим на пример. Банда Twice Charred на вымышленном (в основном) сайте обзора фильмов, где четыре хороших друга и обозревателя фильмов, Лори, Марти, Тори и Курц смотрят фильмы и выставляют им оценки по шкале от 0 до 100.

```
# Before we do anything, we need to import NumPy
import numpy as np
```

Когда банда оценивает фильм, мы можем сохранить их оценки в массиве NumPy `movie_ratings`:

```
movie_ratings = np.array([63.0, 54.0, 70.0, 50.0])
```

Но они смотрят больше одного фильма, поэтому нам нужно создать двумерный массив, в каждой строке которого указаны их рейтинги для определенного фильма.

```
movie_ratings = np.array([[63.0, 54.0, 70.0, 50.0],  
                          [94.0, 85.0, 89.0, 95.0],  
                          [64.0, 90.0, 73.0, 85.0]])
```

Некоторые фанаты предпочитают, чтобы фильмы оценивались по пятизвездочной шкале, поэтому мы можем использовать NumPy, чтобы легко разделить каждый элемент на 20.

```
movie_ratings_stars = movie_ratings / 20
```

Теперь предположим, что рейтинги всегда находятся в одном и том же порядке (Лори, Марти, Тори, Курц), если мы хотим создать массив, в котором были бы только рейтинги Тори, мы могли бы выбрать его из нашего массива `movie_ratings`.

```
tori_ratings = movie_ratings[:, 2]  
tori_ratings  
Out: array([ 70.,  89.,  73.]
```

Теперь предположим, что мы обнаруживаем, что у нас очень похожие вкусы на Марти, поэтому мы хотим смотреть только те фильмы, которым он дает хорошую оценку, мы можем использовать логику для выбора этих фильмов.

Выберем все оценки Марти, превышающие 80:

```
marty_ratings = movie_ratings[:, 1]  
marty_ratings[marty_ratings > 80]  
Out: array([ 85.,  90.]
```

Импорт NumPy

Чтобы использовать NumPy с Python, импортируйте его в начало файла, используя следующую строку:

```
import numpy as np
```

Запись как `np` позволяет нам использовать `np` как сокращение для NumPy, что экономит время при вызове функции NumPy (меньше ввода = меньше ошибок!)

Задание

1. Импортируйте пакет NumPy

Массивы NumPy

NumPy включает мощную структуру данных, известную как массив. Массив NumPy — это особый тип списка. Это структура данных, которая объединяет несколько элементов. Каждый элемент может быть любого типа (строки, числа или даже другие массивы).

Массивы наиболее эффективны, когда они используются для хранения чисел. Это связано с тем, что массивы дают нам особые способы выполнения математических операций, которые проще писать и более эффективны с точки зрения вычислений. Мы поговорим об этом позже.

Массив NumPy очень похож на список Python:

```
my_array = np.array([1, 2, 3, 4, 5, 6])
```

Мы можем преобразовать обычный список в массив NumPy, используя `np.array ()` и сохранив значение в новой переменной:

```
my_list = [1, 2, 3, 4, 5, 6]
my_array = np.array(my_list)
```

Задание

1. Представьте, что вы учитель и вам нужно следить за результатами тестов своего ученика. По первому тесту ученики получили следующие баллы:

92, 94, 88, 91, 87

Создайте массив NumPy с этими значениями и сохраните его под именем `test_1`.

Создание массива из CSV

Как правило, данные непосредственно в массив не вводятся. Вместо этого вы будете импортировать данные откуда-то еще.

Мы можем преобразовывать файлы CSV (значения, разделенные запятыми) в массивы с помощью функции `np.genfromtxt ()`:

Рассмотрим следующий CSV-файл `sample.csv`,

```
34,9,12,11,7
```

Мы можем импортировать это в массив NumPy, используя следующий код:

```
csv_array = np.genfromtxt('sample.csv', delimiter=',')
```

Обратите внимание, что в этом случае в нашем файле `sample.csv` есть значения, разделенные запятыми, поэтому мы используем `delimiter = ','`, но иногда вы можете найти файлы с другими разделителями, наиболее распространенными из которых являются табуляции или двоеточия.

После импорта этот CSV создаст массив

```
>>> csv_array
array([34, 9, 12, 11, 7])
```

Задание

Вы обнаружили, что ввод результатов учащихся в массив вручную занимает слишком много времени. Импортируйте результаты учащегося по второму тесту из CSV test_2.csv (создайте файл самостоятельно, в котором будут содержаться данные: 79, 100, 86, 93, 91) в массив. Сохраните массив с именем test_2.

Операции с массивами NumPy

Как правило, массивы NumPy более эффективны, чем списки. Одна из причин заключается в том, что они позволяют выполнять поэлементные операции. Поэлементная операция позволяет быстро выполнять операцию, например сложение, над каждым элементом массива.

Давайте сравним, как добавить число к каждому значению в списке Python и массиве NumPy:

```
# With a list
l = [1, 2, 3, 4, 5]
l_plus_3 = []
for i in range(len(l)):
    l_plus_3.append(l[i] + 3)
# With an array
a = np.array(l)
a_plus_3 = a + 3
```

Как мы видим, если бы мы добавили 3 к каждому числу в списке, нам пришлось бы использовать цикл for или понимание списка. С массивом мы можем просто добавить 3. То же самое верно для вычитания, умножения и деления.

Мы также можем использовать массивы NumPy, чтобы найти квадраты или квадратные корни каждого значения.

Возведение каждого значения в квадрат:

```
>>> a ** 2
array([ 1,  4,  9, 16, 25, 36])
```

(Примечание: ** - это обозначение степени в Python. Например, 3 в квадрате можно вычислить с помощью 3 ** 2.)

```
>>> np.sqrt(a)
array([ 1,  1.41421356,  1.73205081,  2,  2.23606798,  2.44948974])
```

Задание

1. Оценки учащегося по третьему тесту хранятся в массиве test_3.

Но оказывается, что в одном из вопросов третьего теста была ошибка. Дайте каждому студенту два дополнительных балла и сохраните новый массив в `test_3_fixed`.

```
test_3 = np.array([87, 85, 72, 90, 92])
```

Операции с массивами NumPy II

Массивы также могут быть добавлены или вычтены друг из друга в NumPy, при условии, что массивы имеют одинаковое количество элементов.

При добавлении или вычитании массивов в NumPy каждый элемент будет добавлен / вычтен к соответствующему элементу.

```
>>> a = np.array([1, 2, 3, 4, 5])
>>> b = np.array([6, 7, 8, 9, 10])
>>> a + b
array([ 7,  9, 11, 13, 15])
```

Задание

1. Давайте найдем среднее значение результатов тестов каждого учащегося, чтобы рассчитать их итоговую оценку за семестр. Начните с добавления трех массивов и сохраните ответ в переменной `total_grade`. Не забудьте использовать измененные баллы для третьего теста.
2. Теперь разделите `total_grade` на количество пройденных тестов, чтобы найти средний балл для каждого ученика. Сохраните ответ в переменной `final_grade`.
3. Выведите результат в консоль

Двумерные массивы

В Python мы можем создавать списки, состоящие из других списков. Точно так же в NumPy мы можем создать массив массивов. Если все массивы, составляющие наш больший массив, имеют одинаковый размер, то у него есть специальное имя: двумерный массив.

В предыдущих упражнениях мы хранили результаты тестов студентов в отдельных одномерных массивах для каждого теста:

```
test_1 = np.array([92, 94, 88, 91, 87])
test_2 = np.array([79, 100, 86, 93, 91])
test_3 = np.array([87, 85, 72, 90, 92])
```

Но мы могли бы также сохранить все эти данные в одном двумерном массиве:

```
np.array([[92, 94, 88, 91, 87],
          [79, 100, 86, 93, 91],
          [87, 85, 72, 90, 92]])
```

Здесь каждая строка представляет тест, а каждый столбец представляет учащегося. Это позволяет нам хранить все наши данные в одном массиве без потери какой-либо структуры.

Как мы уже упоминали, двумерный массив — это список списков, каждый из которых имеет одинаковое количество элементов. Вот несколько примеров, которые не являются двумерными массивами.

Этот код будет запущен, но он не создаст двумерный массив, потому что списки имеют разное количество элементов:

```
np.array([[29, 49, 6],
          [77, 1]])
```

Этот код не будет выполнен, потому что [] для внешних списков отсутствует:

```
np.array([68, 16, 73],
          [61, 79, 30])
```

Задание

1. В статистике мы часто используем двумерные массивы для представления набора выборок. Например, если мы подбрасываем монету, мы можем представить каждую орел как 1, а каждый решка как 0.

Создайте одномерный массив для эксперимента с подбрасыванием монеты, который приводит к получению орлов, решек, решек, орлов, решек, и сохраните его в переменной coin_toss.

2. Мы снова запускаем эксперимент и получаем следующий результат: решка, решка, орел, орел, орел. Создайте новый массив, который представляет оба результата как один эксперимент. Сохраните новый массив в coin_toss_again.

Выбор элементов из одномерного массива

NumPy позволяет нам выбирать элементы из массива, используя их индексы. Рассмотрим одномерный массив

```
a = np.array([5, 2, 7, 0, 11])
```

Если бы мы хотели выбрать первый элемент в этом массиве, мы бы вызывали:

```
>>> a[0]
5
```

В типичной манере Python индексы для массива начинаются с 0. Это называется нумерацией с нулевым индексом. В приведенном выше массиве 5 является нулевым элементом a[0]. Отсюда следует, что 2 - это первый элемент a[1].

Мы также можем выбрать отрицательные индексы, которые отсчитываются с противоположного конца массива и начинаются с -1. Это особенно полезно, когда вы хотите получить доступ к последнему или двум элементам массива:

```
>>> a[-1]
11
>>> a[-2]
0
```

Если мы хотим выбрать несколько элементов в массиве, мы можем определить диапазон, например [1: 3], который выберет все элементы от [1] до [3], включая [1], но исключая a [3].

```
>>> a[1:3]
array([2, 7])
```

Точно так же, если бы мы хотели выбрать все элементы перед [3], мы бы использовали:

```
>>> a[:3]
array([5, 2, 7])
```

Мы также можем использовать отрицательные индексы для выбора нескольких элементов. Допустим, мы хотим выбрать последние 3 элемента в массиве:

```
>>> a[-3:]
array([7, 0, 11])
```

Обратите внимание: когда мы выбираем несколько элементов, мы получаем массив.

Задание

1. Вернемся к результатам тестов наших учеников. В следующей таблице показаны все три массива тестов, выровненные по именам учащихся.

	Tanya	Manual	Adwoa	Jeremy	Cody
test_1	92	94	88	91	87
test_2	79	100	86	93	91
test_3	87	85	72	90	92

Джереми хочет знать, сколько он набрал во втором тесте.

Выберите результат из массива test_2 и сохраните его в переменной jeremy_test_2.

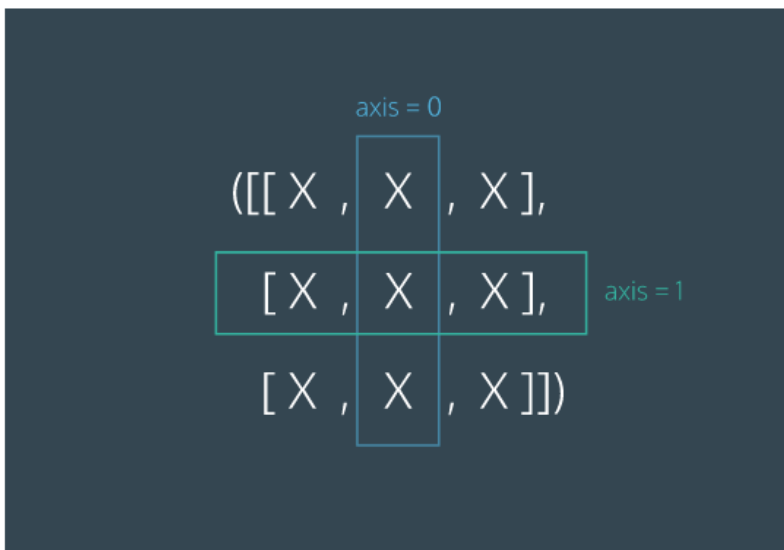
2. Вы хотите сравнить, как справились Manual и Adwoa в первом тесте.

Выберите оба их результата и сохраните их в массиве с именем manual_adwoa_test_1.

Выбор элементов из двумерного массива

Выбор элементов из двумерного массива очень похож на выбор их из одномерного массива, у нас просто есть два индекса для выбора. Синтаксис для выбора из двумерного массива — это [строка, столбец], где а - это массив.

Важно отметить, что, когда мы работаем с массивами, имеющими более одного измерения, отношения между внутренними массивами определяются в терминах осей. Двумерный массив имеет две оси: ось 0 представляет значения, которые имеют одну и ту же индексную позицию (находятся в одном столбце), а ось 1 представляет значения, которые совместно используют массив (находятся в одной строке). Это проиллюстрировано ниже.



Рассмотрим массив

```
a = np.array([[32, 15, 6, 9, 14],
              [12, 10, 5, 23, 1],
              [2, 16, 13, 40, 37]])
```

Мы можем выбрать определенные элементы, используя их индексы:

```
>>> a[2,1]
16
```

Допустим, мы хотели выделить весь столбец, мы можем вставить: в качестве индекса строки:

```
# selects the first column
>>> a[:,0]
array([32, 12, 2])
```

То же самое работает, если мы хотим выделить всю строку:


```
# selects the second row
>>> a[1,:]
array([12, 10,  5, 23,  1])
```

Мы можем еще больше сузить его и выбрать диапазон из определенной строки:

```
# selects the first three elements of the first row
>>> a[0,0:3]
array([32, 15,  6])
```

Задание

1. Результаты тестов наших студентов теперь хранятся в двумерном массиве `student_scores`. В первой строке хранятся результаты первого теста, во второй строке - второго теста, а в третьей строке - третьего теста, как показано в следующей таблице:

	Tanya	Manual	Adwoa	Jeremy	Cody
test_1	92	94	88	91	87
test_2	79	100	86	93	91
test_3	87	85	72	90	92

```
student_scores = np.array([[92, 94, 88, 91, 87],
                           [79, 100, 86, 93, 91],
                           [87, 85, 72, 90, 92]])
```

Таня хочет знать, насколько хорошо она сдала третий тест. Выберите ее результат из массива и сохраните его в `tanya_test_3`.

2. У вас запланировано родительское собрание с родителями Коди, и вы хотели бы иметь под рукой все его результаты тестов.

Выберите все результаты тестов Коди и сохраните их в новом массиве `cody_test_scores`.

Логические операции с массивами

Еще одна полезная вещь, которую могут делать массивы, — это выполнять поэлементные логические операции. Например, предположим, что мы хотим знать, сколько элементов в массиве больше 5. Мы можем легко написать код, который проверяет наличие значения `True` для каждого элемента в массиве, без использования цикла `for`:

```
>>> a = np.array([10, 2, 2, 4, 5, 3, 9, 8, 9, 7])
>>> a > 5
```

```
array([True, False, False, False, False, False, True, True, True,
       True], dtype=bool)
```

Затем мы можем использовать логические операторы для оценки и выбора элементов на основе определенных критериев. Чтобы выбрать все элементы из предыдущего массива, которые больше 5, мы должны написать следующее:

```
>>> a[a > 5]
array([10, 9, 8, 9, 7])
```

Мы также можем комбинировать логические утверждения для дальнейшего уточнения наших критериев. Для этого мы помещаем каждый оператор в круглые скобки и используем логические операторы, такие как & (и) и | (или).

В нашем примере мы можем использовать комбинированные операторы, чтобы найти элементы, которые больше пяти или меньше двух:

```
>>> a[(a > 5) | (a < 2)]
array([10, 9, 8, 9, 7])
```

Задание

1. Сегодня мы посещаем Goldilocks Porridge Festival, пробуем различные хлопья для завтрака и оцениваем их по температуре (указывается в градусах Фаренгейта).

Создайте логическое условие, которое выбирает образцы в массиве овсянки меньше 60 и сохраняет их в переменной с именем cold.

```
porridge = np.array([79, 65, 50, 63, 56, 90, 85, 98, 79, 51])
```

2. Создайте логическое условие, которое находит все значения, превышающие 80, и сохраняет их в переменной с именем hot.
3. Создайте логическое условие, которое находит все значения в диапазоне от 60 до 80 (включительно) и сохраняет их в переменной с именем just_right.
4. Выведите в консоль каждый массив

Заключение

В этом уроке вы узнали основы пакета NumPy. Вот несколько ключевых моментов:

- Массивы - это особый тип списка, который позволяет нам хранить значения в организованном порядке.
- Массив можно создать, определив его напрямую с помощью np.array () или импортировав CSV с помощью np.genfromtxt ('file.csv', delimiter = ',').
- Операция (например, сложение) может выполняться с каждым элементом массива, просто выполняя ее над самим массивом.

- Элементы могут быть выбраны из массивов, используя их индекс и расположение в массиве, оба из которых начинаются с 0.
- Логические операции могут использоваться для создания новых, более сфокусированных массивов из более крупных массивов.

Задание

1. Вы работаете в группе ученых-климатологов, и одна из ваших ролей - принимать еженедельные данные о температуре и вводить их в массивы NumPy для последующего анализа. Датчик регистрирует температуру 4 раза в день в 0:00, 6:00, 12:00 и 18:00. Вам предоставляются данные за последние недели (с понедельника по пятницу) и вас просят создать массив NumPy.

Начните с импорта пакета numpy.

2. Создайте массив температур, импортировав данные в CSV-файл temperature_data.csv (CSV на основе данных Национального управления океанических и атмосферных исследований). CSV содержит данные:

```
43.6, 45.1, 58.8, 53
47, 44, 58.3, 52.6
46.7, 44.2, 57.9, 52.2,
46.5, 44.1, 57.6, 51.9,
46.2, 43.9, 57.2, 51.5
```

3. Проверьте данные, выведя их в терминал.

Столбцы — это время, начиная с 0:00, а строки - это дни, начиная с понедельника, а значения представляют собой зарегистрированные температуры в это время в эти дни.

4. Один из исследователей заметил, что датчик был неправильно обнулен, и все зарегистрированные температуры были на 3,0 градуса ниже нуля.

Исправьте массив так, чтобы показания температуры были точными, и сохраните их в Temperature_fixed.

5. Другой исследователь спросил у вас значения температуры понедельника. Сохраните их в новом массиве monday_temperas.
6. «Хммм, интересно, - бормочет исследователь, - можешь также дать мне данные за 6:00 утра по четвергам и пятницам?»
Сохраните эти данные в новом массиве thursday_friday_morning.
7. Наконец, настойчивый исследователь теперь хочет, чтобы в течение недели были все высокие и низкие температуры.

Выберите все температуры ниже 50 или выше 60 градусов и сохраните их в новом массиве temperature_extremes.